Networking TS Issues

- Document number: **P0703R0**, ISO/IEC JTC1 SC22 WG21
- Date: 2017-06-18

- Authors: David Sankel dsankel@bloomberg.net

- Audience: LEWG

Abstract

This paper provides elaboration of US9 and US10 ballot comments on the Networking TS as requested by LEWG at the Kona 2017 meeting. In summary, Bloomberg formed a committee of networking experts to discuss the Networking TS and concluded that there were a couple usability concerns that should be addressed.

US9

Original text:

> The `CompletionToken` mechanism used to determine the return types of initiating functions has teachability drawbacks due to its complexity. Also, there would be a high cost if users were to replicate this kind of interface in medium and higher-level libraries built upon the Networking TS. A simpler interface whereby initiating functions have invokable parameters is sufficiently extendable, as they are, to work with futures.

Suggested resolution:

> Make initiating functions use invokable callbacks instead of completion tokens as in Boost.ASIO.

Asynchronous operations (called initiating functions) in the Networking TS follow a pattern where a `CompletionToken` parameter controls both the return value of the function and how the calling code is notified of the operation's completion. For example, see the specification of one of the `async_connect` overloads:

```
template<class Protocol, class InputIterator,
class ConnectCondition, class CompletionToken>
DEDUCED async_connect(basic_socket<Protocol>& s,
                      const EndpointSequence& endpoints,
                      ConnectCondition c,
                      CompletionToken&& token);
```

The `CompletionToken`-related behavior is specified as follows:

> An initiating function determines the type `CompletionHandler` of its completion handler function object by performing `typename async_result<decay_t<CompletionToken>, Signature>::completion_handler_type`. The completion handler object `completion_handler` is initialized with `std::forward<CompletionToken>(token)`.

, where,

> The `async_result` class template is a customization point for asynchronous operations. Template parameter `CompletionToken` specifies the model used to obtain the result of the asynchronous

> operation. Template parameter `Signature` is the call signature (C++Std [func.def]) for the completion handler type invoked on completion of the asynchronous operation

.

If the `CompletionToken` is a callback, the asynchronous operation will simply schedule a call to the callback with the result of the operation. If the `CompletionToken` is `use_future`, then the result of the operation will be a `std::future` object that is fulfilled with result of the operation (or rejected with an exception object that includes the error). The benefits of this approach are two-fold: with one function we support both callbacks and futures, and we provide an extension mechanism for supporting any other future-callback-like mechanism.

While this is a clever way to solve the problem, we feel that having a simple callback would be superior. First, the mechanism by which the completion token influences the behavior of the function is difficult to understand and explain. Second, plain callbacks, as the simplest way to signal asynchronous completions, are sufficient for building future-based interfaces on top.

Overall, we feel that a lower-level, and much simpler, callback interface would better serve the needs of the C++ and networking community.

US10

Original Text:

> Without knowing more about what kind of executor is being used, a user will have difficulty deciding which of the three functions to use to add a task to an executor. It is preferable to limit the options for adding tasks to a generic executor. Concrete executors can add additional functions if required.

Suggested Resolution:

> Remove the defer function from executors, as that is the least well-defined. This would match the existing Boost ASIO implementation.

The current Networking TS proposal has three ways to schedule a "task" with an executor: `dispatch`, `post`, and `defer`. To paraphrase section 13.2.2:

`dispatch(f)`
: May call `f` immediately and wait for it to complete before `dispatch` returns, otherwise schedule `f` to be executed.

`post(f)` and `defer(f)`
: Schedule `f` to be executed. Neither `post` nor `defer` may block on `f`'s completion.

While `post` and `defer` have identical semantics, the following note is adjoined in the Networking TS text:

> Note: Although the requirements placed on `defer` are identical to post, the use of `post` conveys a preference that the caller does not block the first step of `f1`'s progress, whereas `defer` conveys a preference that the caller does block the first step of `f1`. One use of `defer` is to convey the intention of the caller that `f1` is a continuation of the current call context. The executor may use this information to optimize or otherwise adjust the way in which `f1` is invoked.

The idea is that the writer of a component who is aware of the executor being used can squeeze out some additional performance by choosing `defer` or `post` correctly.

We find the difference between `post` and `defer` to be too subtle and too executor-specific to be useful in the C++ standard. The confusion caused outweighs the performance gain in the Bloomberg committee's opinion.

Additionally, it has not been demonstrated that performance gains from having a distinction between `post` and `defer` make a difference in any real-world applications.

For applications that really need a `post` and `defer` distinction, we suggest use of a custom executor class that provides the additional operations; these applications are specialized enough to warrant knowledge of the specific executor they are used with.

Conclusion

In summary, we feel the Networking TS should be simplified in the areas of completion tokens and executor interface to best serve the C++ community.